

So sánh MVC cũ vs Web API mới

TECH-SPECTRUM

1. Kiến trúc tổng thể

	MVC cũ	Web API mới
View	PHP render HTML (app/views/)	HTML tĩnh (frontend/)
Controller	Xử lý logic + trả về View	Chỉ trả về JSON
Model	Giữ nguyên	Giữ nguyên
Response	HTML page	JSON data
URL ví dụ	/product/list → load trang mới	/api/product → trả JSON
Xác thực	\$_SESSION	JWT Token

2. Luồng xử lý

MVC cũ

Browser → Request → PHP Controller → render View (HTML) → Browser hiển thị

Web API mới

Browser (HTML+JS) → fetch() → PHP API Controller → JSON → JS cập nhật DOM

3. Điểm khác biệt chính

Xác thực (Authentication)

- MVC: dùng \$_SESSION — server lưu trạng thái giữa các request, mỗi request gửi kèm cookie
- API: dùng JWT Token — server không lưu trạng thái, client tự mang token theo mỗi request (stateless)

Phân quyền (Authorization)

- MVC: check session trong PHP, header('Location: ...') nếu không có quyền
- API: check JWT → trả 401 Unauthorized hoặc 403 Forbidden dùng JSON, frontend tự redirect

Giao diện (Frontend)

- MVC: mỗi hành động reload lại trang mới (full page reload)
- API: trang không reload, JavaScript gọi API rồi cập nhật giao diện (SPA-like)

Tái sử dụng

- MVC: chỉ phục vụ web browser
- API: cùng 1 backend phục vụ web, mobile app, ứng dụng bên thứ ba

Tách biệt Frontend – Backend

- MVC: frontend và backend dính liền, PHP sinh ra HTML

- API: frontend và backend hoàn toàn độc lập, có thể deploy riêng, team làm việc song song

4. Ví dụ cụ thể – Thêm sản phẩm

MVC cũ (PHP)

```
// ProductController.php
public function add() {
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        // lưu vào DB
        header('Location: /product/list'); // redirect -> load lại trang
        exit;
    }
    include 'app/views/product/add.php'; // render HTML
}
```

Web API mới (PHP + JS)

```
// ProductApiController.php
public function store(): void {
    $this->requireAdmin(); // kiểm tra JWT role=admin
    $data = $this->getBody(); // đọc JSON từ request body
    // lưu vào DB
    $this->json(['message' => 'Thêm thành công', 'id' => $id], 201);
}

// Frontend JS (admin/products.html)
const res = await POST('/product', { name, price, category_id });
if (res.ok) showToast('Thêm sản phẩm thành công');
```

5. Cấu trúc thư mục

MVC cũ

```
app/
├── controllers/
│   ├── ProductController.php <- xử lý + render view
│   └── AuthController.php
├── models/
│   └── ProductModel.php
└── views/
    ├── product/list.php <- HTML do PHP sinh ra
    └── layouts/header.php
```

Web API mới

```
app/
├── controllers/
│   ├── ApiController.php <- base class (JSON, JWT)
│   ├── ProductApiController.php <- xử lý JSON
│   └── AuthApiController.php
├── models/
│   └── ProductModel.php <- giữ nguyên
├── helpers/
│   └── JwtHelper.php <- tạo/validate JWT
└── frontend/ <- hoàn toàn tách biệt
    ├── js/api.js <- fetch wrapper + Auth
    ├── index.html
    ├── login.html
    └── admin/
        └── products.html
```

6. HTTP Status Code trong Web API

Code	Ý nghĩa	Ví dụ
200 OK	Thành công	GET danh sách sản phẩm
201 Created	Tạo mới thành công	POST thêm sản phẩm
400 Bad Request	Dữ liệu không hợp lệ	Thiếu trường bắt buộc
401 Unauthorized	Chưa đăng nhập / token sai	Không có Bearer token
403 Forbidden	Không có quyền	User gọi API admin
404 Not Found	Không tìm thấy	ID sản phẩm không tồn tại
500 Internal Server Error	Lỗi server	Lỗi DB, lỗi upload file

7. Ưu – Nhược điểm

MVC cũ

Ưu điểm	Nhược điểm
Đơn giản, dễ học	Không tái sử dụng cho mobile
Ít file html	Reload trang gây chập
Server render → SEO tốt hơn	Frontend – Backend phụ thuộc nhau

Web API mới

Ưu điểm	Nhược điểm
Tái sử dụng cho nhiều client	Phức tạp hơn (JWT, CORS)
Trải nghiệm mượt mà (không reload)	Cần JavaScript nhiều hơn
Frontend – Backend độc lập	SEO kém hơn nếu không có SSR
Dễ scale và bảo trì	